EVOLUTIONARY DISCOVERY OF REINFORCEMENT LEARNING ALGORITHMS VIA LARGE LANGUAGE MODELS

Sygkounas, Loutfi & Persson — GECCO 2026

J.C. Vaught
CSCE 775 — Paper Presentation
University of South Carolina

INTRODUCTION

Framing the problem

WHAT RL IS USED FOR

Reasoning LLMs

o1, DeepSeek-R1 — RL makes them think

LLM alignment

RLHF, DPO, GRPO, Constitutional AI

Robotics & autonomy

π_0, OpenVLA, Atlas, Waymo, AlphaStar

Science & operations

AlphaDev, AlphaTensor, chip design, data-center cooling

THE PROBLEM WITH RL & ES

Reinforcement Learning

+ sample-efficient per step: SAC $\approx$ 100K steps on MuJoCo
+ gradient signal $\Rightarrow$ credit assignment over long horizons

— still needs 10^6–10^7 env steps (humans: $\sim 10^2$)

— hyperparameter swings change return by $\mathbf{3}\times$ (Henderson '18)

— reward hacking and instability; reproducibility crisis

Evolutionary Search

+ scales linearly to $\mathbf{1{,}440}$ workers (Salimans '17)
+ humanoid solved in $\mathbf{10}$ min wall-clock vs. hours for PPO

— uses $\mathbf{3}$–$\mathbf{10}\times$ more env steps than A3C/PPO

— struggles past $\sim 10^6$ parameters (no gradient info)

— search space must be hand-designed (DSL / grammar)

Each is strong where the other is weak — neither alone is positioned to invent new algorithms.

THE CLAIM

First paper to evolve entire RL update rules as executable Python code using an LLM as the variation operator.

The idea. A candidate algorithm is a Python `compute_loss` function. The LLM mutates and recombines code; real training runs score it. Actor–critic, value functions, and Bellman updates are banned in the prompt — so only genuinely new mechanisms survive.

Lineage. AutoRL had tried this with hand-crafted DSLs (Co-Reyes '21) or black-box neural nets (LPG, DiscoRL) — both boxed the search space in. LLM-code-evolution (FunSearch, AlphaEvolve) hit big math wins, and REvolve evolved rewards — but never the algorithm itself.

My take. Impressive proof-of-concept — but compute cost, narrow benchmarks, and the strong possibility that the LLM is remembering more than it's discovering leave real problems on the table.

MODERN RL RESTS ON 5 RULES

DQN	TRPO	DDPG	PPO	SAC	TD3
2015	2015	2015	2017	2018	2018

A decade of progress — hand-designed update rules.

Each took months of PhD-level effort. Each became dogma.

AUTOML CLIMBS THE LADDER

The update rule itself	???
Preference loss	DiscoPOP
Reward function	Eureka, REvolve
Hyperparameters	PB2, OptFormer
Architecture	NAS, AmoebaNet

The loss function was the last frontier.

BACKGROUND

Two lineages converge on one idea

LINEAGE A: DISCOVER AN RL ALGORITHM

- **EPG ’18** — neural loss for PG
- **LPG ’20** — LSTM update rule
- **Co-Reyes ’21** — DAG over typed ops
- **DiscoRL ’25** — meta-learned NN (Nature)

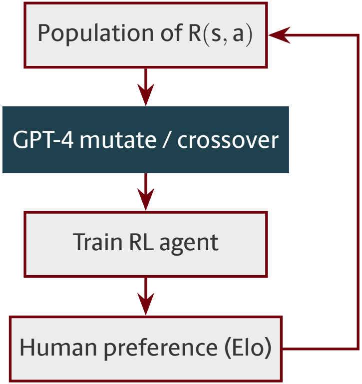
All constrain the search space by hand.

LINEAGE B: LLMS MUTATE CODE

- **ELM ’22** — LLMs as mutators
- **FunSearch ’24** — cap-set, bin-packing
- **AlphaEvolve ’25** — full files, Strassen beaten
- **EvoTune ’25** — proposer fine-tuning

> LLMs are intelligent variation operators over code.

PARENT WORK: REVOLVE (2024)



What REvolve evolves

- Executable Python reward functions
- Island populations, LLM variation
- Tested on driving, humanoid, dexterous hands

The move in this paper.
Evolve the loss function instead of the reward.

THE GAP THIS PAPER FILLS

What gets evolved		Representative method
Architecture	✓	NAS, AmoebaNet
Hyperparameters	✓	PB2, OptFormer
Reward function	✓	REvolve, Eureka
Preference loss (align)	✓	DiscoPOP
Update rule (DAG)	✓	Co-Reyes 2021
Update rule (black-box NN)	✓	LPG, DiscoRL
Update rule (executable Python, LLM-evolved)	★	Sygkounas et al. 2026

Everything above is context. The bottom row is the thesis.

METHOD

How the framework works

THE BIG PICTURE

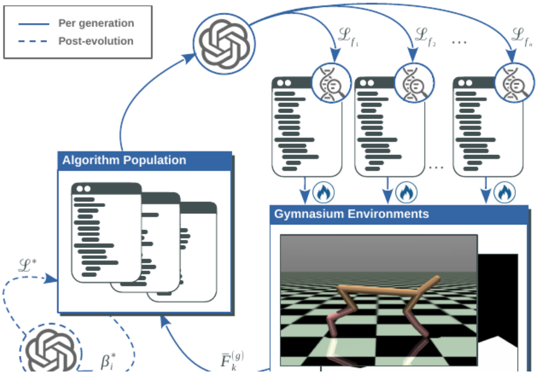


Figure 1 from the paper.

Per generation

- LLM proposes candidate loss functions
- Each is trained end-to-end on 5 Gym envs
- Fitness drives selection

Post-evolution

- LLM proposes hyperparameter ranges
- Sweep finds the refined rule $\mathcal{L}^\star$

A CANDIDATE IS A CHUNK OF PYTHON

Parent

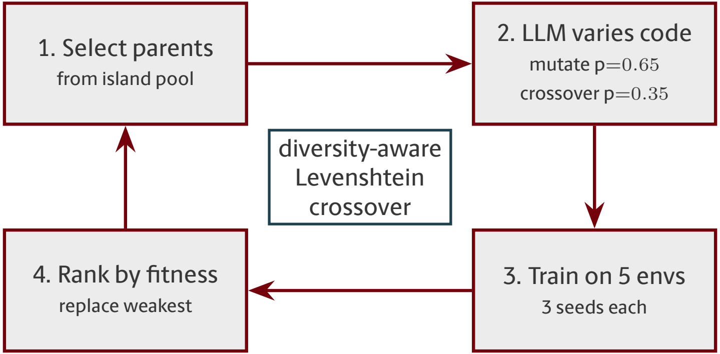
```
def compute_loss(theta, xi, D):
  preds = model(D.s, theta)
  targets = rollout(model, D.s)
  L = mse(preds, targets)
  return L
```

After LLM mutation

```
def compute_loss(theta, xi, D):
  preds = model(D.s, theta)
  conf = xi.confidence(D.s)
  targets = rollout(model, D.s)
  L = (conf * mse(preds, targets)).mean()
  return L
```

The LLM edits Python. That is the entire search space.

THE EVOLUTIONARY LOOP



Island model · 9 individuals per island · 24 new candidates per generation.

WHAT THE LLM IS TOLD NOT TO DO

No actor–critic decomposition

No bootstrapped value targets

No temporal-difference updates

Constitutional prompt constraints $\Rightarrow$ forces novel mechanisms.

Evolution cannot re-invent DQN; it has to find something else.

LLM-GUIDED HPO PASS

Before LLM-HPO

Evolved rule works
but fitness is noisy across
hyperparameter choices.

→

After LLM-HPO

LLM proposes bounded
interval $[\ell_j, u_j]$ per scalar.
Sweep $\Rightarrow$ refined rule $\mathcal{L}^\star$.

Policy arch, optimizer, rollout loop stay frozen. Only internal scalars move.

RESULTS

What evolution discovered

THE TWO ALGORITHMS IT INVENTED

CG-FPD

Confidence-Guided Forward Policy Distillation

Distill short-horizon plans from a learned latent dynamics model. Planning acts as the supervised teacher signal.

No value function. No policy gradient. No Bellman.

DF-CWP-CP

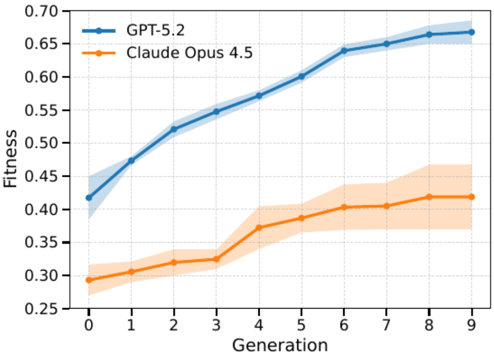
Differentiable Forward Confidence-Weighted Planning with Controllability Prior

Differentiable world-model rollouts + confidence-weighted desirability + controllability prior.

Also avoids every canonical RL mechanism.

Evolution rediscovered model-based planning — from scratch.

THE PROPOSER LLM MATTERS. A LOT.



- GPT-5.2 climbs to 0.67
- Claude 4.5 Opus caps at 0.42
- Both monotone; GPT-5.2 is sharper on every generation

Same framework, same envs, same seeds — just swap the LLM.

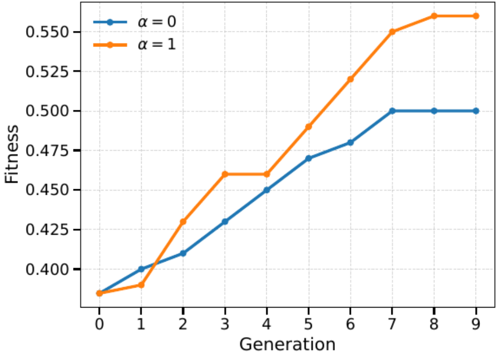
RESULTS MATCH PPO AND SAC

Environment	PPO	SAC	CG-FPD	DF-CWP-CP
CartPole	500.0	—	500.0	500.0
LunarLander	246.6	—	241.2	260.6
MountainCar	-128.1	—	-105.8	-108.6
Acrobot	-63.5	—	-90.6	-78.6
HalfCheetah	1579.1	4988.5	2407.9	2103.8
Swimmer	95.0	87.8	247.5	219.8
Walker2d	3163.2	4595.3	1603.8	1297.6
InvertedPendulum	1000.0	1000.0	1000.0	1000.0

Competitive with hand-designed baselines across 10 environments.

Evolved on 5 training envs; remaining 5 are generalization tests.

ABLATION: DIVERSITY WEIGHT



Levenshtein weight α

- $\alpha = 0$: no structural regularization $\Rightarrow 0.50$
- $\alpha = 1$: full regularization $\Rightarrow 0.56$
- $\alpha = 0.5$: 0.65

Some code-similarity nudge helps; too much or too little hurts.

DISCUSSION

Critique and open questions

STRENGTHS VS. WEAKNESSES

Strengths

- First full update-rule discovery via LLM
- Genuinely novel mechanisms (no value, no PG)
- Clean extension of REvolve's machinery
- LLM-HPO handles fitness fragility cleanly

Weaknesses

- 30 h per candidate on $4 \times$A100
- Narrow benchmarks (CartPole / MuJoCo only)
- Is GPT-5.2 discovering or remembering?
- Only 2 evolutionary seeds per LLM
- No head-to-head vs. DiscoRL
- Constitutional constraints never ablated

WHERE THIS PAPER SITS IN THE FIELD

	RL	Alignment
NN-parameterized	DiscoRL (Oh 2025) meta-learned NN	RLHF / DPO (Rafailov 2023) neural reward model
Code	This paper (Sygkounas 2026) evolved Python	DiscoPOP (Lu 2024) LLM-evolved loss

The real contest — evolved source code vs. meta-gradient for RL.

OPEN QUESTIONS

- Surrogate fitness to break the 30 h/candidate wall.
- Lift the constraints — let evolution decide whether to re-use actor–critic.
- Scale to Atari and robotics — generalization is still untested.
- Curriculum over environments — connect to POET, XLand.
- Self-improving proposer — EvoTune-style DPO on the evolved database.

Takeaway. REvolve showed LLMs can evolve rewards. This paper shows they can evolve the entire algorithm. The open question is whether what they discover is genuinely new — or just a good remix of what the model already saw.

THANK YOU!

Questions & Discussion

jvaught@sc.edu